

Topic: AI Search Lab — 8-Puzzle Solver & Tic-Tac-Toe with AI

Comparative Analysis

Sebastian Honorat

Nicolas Choi

Structural Comparison:

The search space of an 8-puzzle is similar to the game tree of tic-tac-toe in that both pursue an optimal route to a goal state. In the 8-puzzle, the optimal solution is the minimal amount of moves to change from the starting configuration to the goal configuration. In tic-tac-toe, the optimal solution is the move that produces the best possible outcome assuming both players play optimally for a winner. It differs in that tic-tac-toe has two players competing against each other and both players are trying to win individually, while the puzzle solver is a single-agent problem.

Algorithm Fit:

As mentioned in the structural comparison, A* is not applicable to Tic-Tac-Toe because it is an algorithm for a single agent. Minimax does not apply to 8-puzzle because it is an adversarial search player. The 8-puzzle game does not have two opposing players so there is no assignable mini or max. Alpha-Beta pruning is more suitable because it can use the scoring system to branches that a rational agent would not consider because it assumes its opponent would make a rational decision. It would not be feasible to use a heuristic such as Manhattan Distance when two agents have competing goals that push each other further away.

Empirical Comparison - Module A:

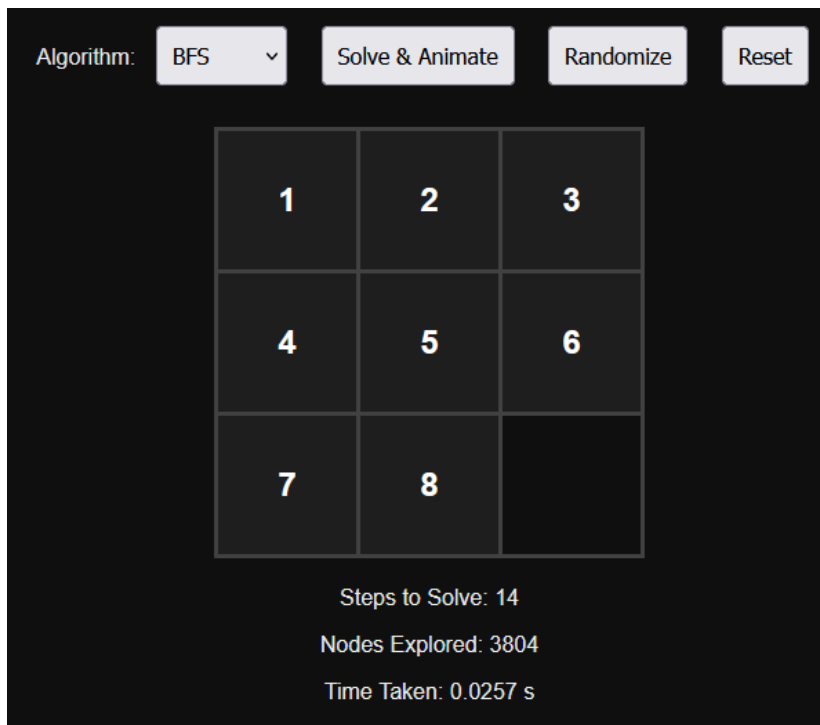


Figure 1. BFS Search Results

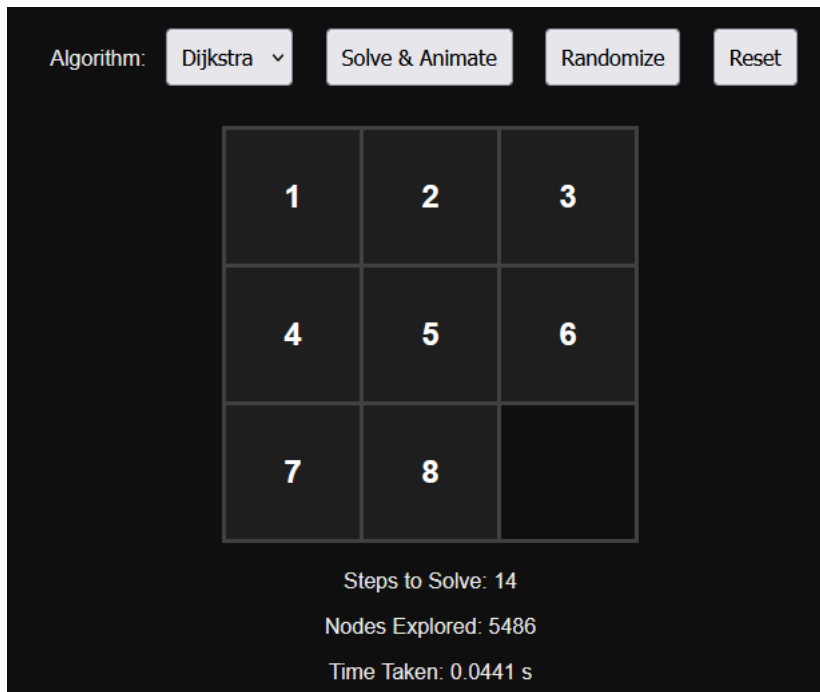


Figure 2. Dijkstra's Search Results

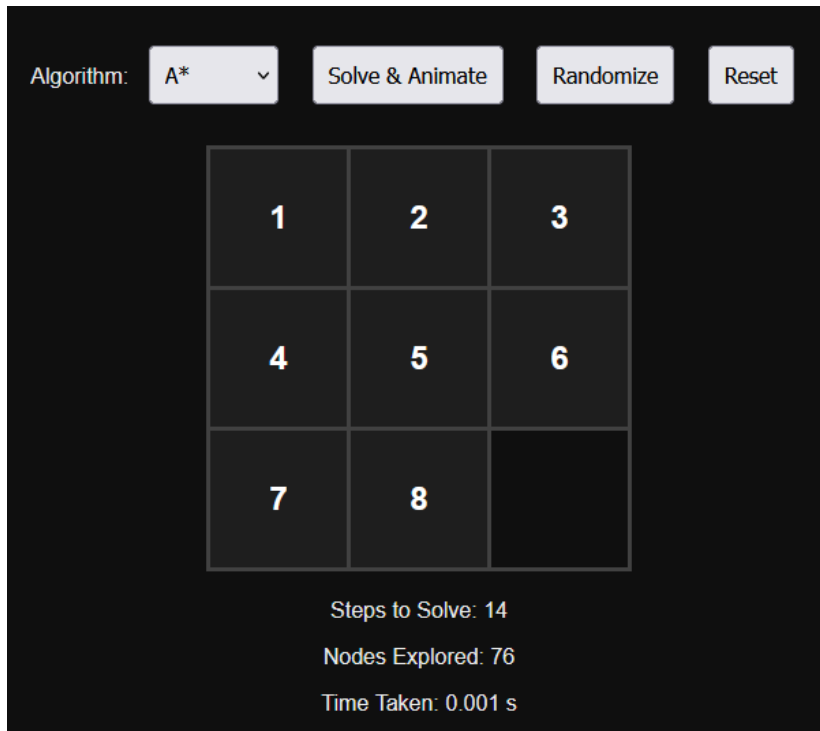


Figure 3. A* Search

Standardized test indicates massive improvement in performance with A* search. The amount of nodes explored dropped significantly from 3804 nodes (fig 1) to 76 nodes (fig 3) which attributes to the value of heuristic information. The Manhattan distance is useful in traversing a search space that will lead to configurations with the fewest moves needed. All algorithms are able to find the shortest path but runtime is by far the shortest with A*.

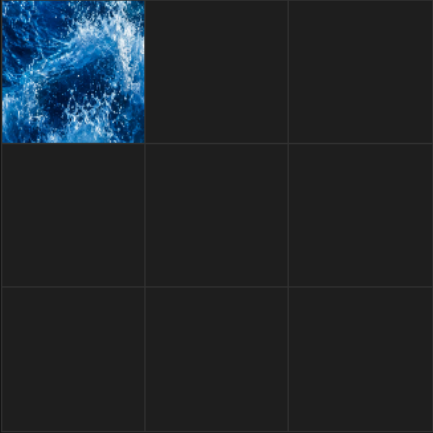
It was surprising to see Dijkstra's algorithm explore the most nodes of 5486 (fig 2). It may be because since the search space is a weighted graph it visits more nodes beyond reaching the goal state therefore looking "deeper" to find the shortest route.

Empirical Comparison - Module B:

Tic Tac Toe AI

[Go to 8 Puzzle Solver](#)

Your Turn (X)



Mode:

AI Algorithm:

Moves: 1

Nodes Explored: 549945

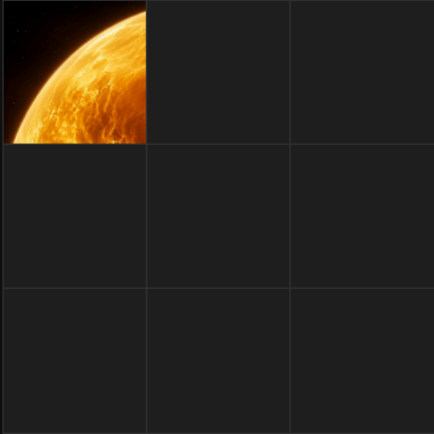
Decision Time: 1.4525

Pruning Rate: --

Tic Tac Toe AI

[Go to 8 Puzzle Solver](#)

Your Turn (X)



Mode:

AI Algorithm:

Moves: 1

Nodes Explored: 28035

Decision Time: 1.6179

Pruning Rate: 94.9%

Alpha-Beta Pruning algorithm reports a 94.9% pruning rate. The search space from minimax has been pruned dramatically for efficiency. It is considered that the decision time is longer because it is performing minimax first to be able to calculate the pruning rate. In practice the decision time is estimated to be about 0.17 second which makes Alpha-Beta pruning almost 9 times faster.

Trade-off-Analysis

BFS

Completeness: BFS is complete because it searches through the entire state space level-by level.

Optimality: BFS is optimal when all moves have the same cost, which is true for the 8-puzzle.

Time Complexity: BFS can be very time consuming especially when the answer is very deep

Space Complexity: BFS consumes a lot of memory storing all nodes at each level with 3804 nodes being costly.

Dijkstra's

Completeness: Dijkstra's is complete because it keeps memory of possible nodes in a priority queue to search an entire state space.

Optimality: Dijkstra is optimal because it finds the least costly path to the solution.

Time Complexity: It can take longer because it keeps track of "weights" in a graph while maintaining a priority queue.

Space Complexity: Dijkstra is using even more memory than BFS keeping track of visited states along with weight.

A*

Completeness: It functions similarly to Dijkstra's while using a heuristic but is guaranteed to visit all nodes.

Optimality: Manhattan Distance is admissible and consistent guaranteeing optimality.

Time Complexity: Heuristic makes A* extremely fast in comparison because it leads to the goal state much more quickly.

Space Complexity: Similar memory usage to Dijkstra keeping track of similar information.

Minimax

Completeness: Minimax can search an entire game tree in a finite space

Optimality: It is optimal when the entire game tree is searched, if plausible an “optimal” metric can be implemented to guarantee optimality further.

Time Complexity: Minimax visits every possible move in a game space which is time consuming

Space Complexity: It can be very costly especially with extremely deep trees such as Tic Tac Toe with roughly 550,000 nodes.

Alpha Beta Pruning

Completeness: Alpha Beta Pruning makes the same decision as Minimax

Optimality: It will not change the root decision and will only save work.

Time Complexity: It is significantly faster from the unnecessary nodes that are pruned and not visited.

Space Complexity: It is typically less intensive than Minimax but dependent on node ordering. In ideal cases, the most amount of nodes that can be pruned are not visited.

Heuristic Justification

Manhattan Distance is the heuristic used for the A* search. It is admissible because it calculates the total of distances each tile is from the goal position. The puzzle cannot be solved in fewer moves than the total Manhattan distance suggests. Thus, the true solution length is never overestimated.